

# leetcode Hot100 CPP题解 个人认为最简单好背的版本

## 1.两数之和

```
class Solution {  
public:  
    vector<int> twoSum(vector<int>& nums, int target) {  
        unordered_map<int,int>m;  
        for(int i=0,j=0;i<nums.size();i++){  
            int r=target-nums[i];  
            if(m.count(r)) return {m[r],i}; //一定要用m.count  
            m[nums[i]]= i;  
        }  
  
        return {};  
    }  
};
```

C++

## 49.字母异位词分组

```
class Solution {  
public:  
    vector<vector<string>> groupAnagrams(vector<string>& strs) {  
        unordered_map<string,vector<string>> hash; //哈希表  
        for(auto &str:strs){  
            string nstr =str; //复制一份  
            sort(nstr.begin(),nstr.end());  
            hash[nstr].push_back(str); //哈希找  
        }  
  
        vector<vector<string>> res;  
        for(auto &item: hash) res.push_back(item.second);  
  
        return res;  
    }  
};
```

C++

## 128.最长连续子序列

C++

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int>m;
        for(auto x:nums){
            m.insert(x); //先插入
        }
        int res=0;
        for(auto x:m){
            if(!m.count(x-1)){//连续的开头不存在
                int t=x;
                int sum=1;
                while(m.count(x+1)){
                    sum++;
                    x++;
                }
                res=max(res,sum);
            }
        }

        return res;
    }
};
```

## 283. 移动零

C++

```
class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int j=0;
        for(int i=0;i<=nums.size()-1;i++){
            if(nums[i]!=0){ //只需要双指针前面的顺序调整好，把后面再置为0就可以
                nums[j++]=nums[i];
            }
        }
        for(;j<=nums.size()-1;j++){
            nums[j]=0;
        }
    }
};
```

## 11. 盛最多水的容器

C++

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int i = 0, j = height.size() - 1, res = 0;
        while(i < j) { //双指针往中间靠
            res = height[i] < height[j] ?
                max(res, (j - i) * height[i++]):
                max(res, (j - i) * height[j--]);
        }
        return res;
    }
};
```

## 15. 三数之和

C++

```
class Solution {
public:
    //n=3000
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;
        //排除特例
        if(nums.size() < 3){
            return result;
        }
        //记得要排序
        sort(nums.begin(),nums.end());
        int n=nums.size();
        //值遍历a为左端点
        for(int i=0, left=i+1,right=n-1; i<n-2 ; i++,left=i+1, right=n-1){

            if(i>0&&nums[i]==nums[i-1]) continue;

            while(left < right){//最左端+最右端 往中间走
                if(nums[i] + nums[left] + nums[right] > 0){//太大了
                    right--;
                }else if(nums[i] + nums[left] + nums[right] < 0){//太小了
                    left++;
                }else{//等于值

                    result.push_back({nums[i],nums[left],nums[right]});
                    //去重 b c
                    while(left < right && nums[left] == nums[left + 1]) left++;
                    while(left < right && nums[right] == nums[right - 1]) right--;

                    left++;//去重还没有完成 继续
                    right--;
                }
            }
        }
        return result;
    }
};
```

## 42.接雨水

C++

```
class Solution {
public:
    int trap(vector<int>& height) {
        int n=height.size();
        if(n==0||n==1) return 0;
        vector<int> l(n+1), r(n+1); //每个高度的统计
        for(int i=1; i<n; i++){ //统计i左边的高度最高的值
            l[i]=max(l[i-1], height[i-1]);
        }
        for(int i=n-2; i>=0; i--){ //统计i右边的高度最高的值
            r[i]=max(r[i+1], height[i+1]);
        }
        int res=0;
        for(int i=0; i<n; i++){
            //每一列的贡献是（两边的高度的最大值的最小值）- 本身的高度
            //如第6列的值 就是第四列-本身的高度 这列答案为2
            res+=(min(l[i], r[i])-height[i])>0 ? (min(l[i], r[i])-height[i]) : 0);
        }
        return res;
    }
};
```

## 3. 无重复字符的最长子串

C++

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        if(!s.size()) return 0;
        set<int> st; //set判重
        int j=0;
        int len=1;
        for(int l=0, r=0; r<s.size(); r++){ //r是右指针 j是左指针
            while(st.count(s[r])){ //有指针遍历到重复的值
                st.erase(s[l]); //有重复的值 说明左指针必须往右跑才能找到新的无重复字符的
                l++;
            }
            st.insert(s[r]); //走到有他的容身之地为止
            len=max(len, r-l+1);
        }
        return len;
    }
};
```

## 438. 找到字符串中所有字母异位词

C++

```
class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        int sLen = s.size(), pLen = p.size();

        if (sLen < pLen) {
            return vector<int>();
        }

        vector<int> ans;
        vector<int> sCount(26);
        vector<int> pCount(26);
        for (int i = 0; i < pLen; ++i) { //对p长度进行处理
            ++sCount[s[i] - 'a'];
            ++pCount[p[i] - 'a'];
        }

        if (sCount == pCount) { //判断两个数组值是不是一样
            ans.emplace_back(0);
        }

        for (int i = 0; i < sLen - pLen; ++i) { //滑动窗口
            --sCount[s[i] - 'a']; //往右边移动一位
            ++sCount[s[i + pLen] - 'a'];

            if (sCount == pCount) { //直接数组比较
                ans.emplace_back(i + 1);
            }
        }

        return ans;
    }
};
```

## 560. 和为 K 的子数组

C++

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> mp;
        mp[0] = 1;
        int count = 0, pre = 0;
        for (auto& x:nums) {
            pre += x;
            //mp[前缀和前面某一段] + K前缀和后边这一段 = pre整体前缀和
            //k这一段是题目要求的
            count += mp[pre - k];

            mp[pre]++;
        }
        return count;
    }
};
```

## 239. 滑动窗口最大值

C++

/每次返回 滑动窗口中的最大值 。

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        int n = nums.size();
        deque<int> q; // 开一个双端队列
        for (int i = 0; i < k; ++i) { // 处理前k个
            while(q.size() && nums[i] > nums[q.back()]) {
                q.pop_back();
                // 单调队列而且求最大值 所以后面进入的比队刚进入的还要大说明队刚进入的这个值
                // 肯定用不上
            }
            // 直接删去
            q.push_back(i); // 注意存的是端点
        }

        vector<int> ans = {nums[q.front()]};
        for (int i = k; i < n; ++i) {
            while(q.size() && nums[i] > nums[q.back()]) {
                q.pop_back();
            }
            q.push_back(i); // 队列存的是递减的值的下表 对头最大
            while(q.front() <= i - k) { // 如果窗口左边界过去, 移出窗口
                q.pop_front();
            }
            ans.push_back(nums[q.front()]); // 对列前面的是答案 因为是最大之
        }
        return ans;
    }
};
```

## 76. 最小覆盖子串

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int>ht,hs; //表示两个字符串的哈希
        for(char c:t) ht[c]++;
        string res;
        int cnt=0;
        for(int l=0,r=0;r<s.size();r++){
            hs[s[r]]++;
            if(hs[s[r]]≤ht[s[r]]) ++cnt; //cnt代表匹配的字符数, 不是字母数
            while(l<r&&hs[s[l]]>ht[s[l]]) hs[s[l]]--; //窗口右移
            if(cnt==t.size()){
                if(res=""||res.size()>r-l+1) res=s.substr(l,r-l+1);
            }
        }
        return res;
    }
};
```

## 最大子数组和

C++

```
//dp 时间On 空间O1
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int maxv=INT_MIN;
        for(int i=0,last=0;i<nums.size();i++){
            last = nums[i] + max(last,0); //last负责新开的一段
            maxv = max(maxv,last); //maxv负责与过去的一段做比较
        }
        return maxv;
    }
};

//求最大子数组是什么版本
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int maxv=INT_MIN;
        vector<int>res,lastve;
        for(int i=0,last=0;i<nums.size();i++){
            vector<int>v;
            bool f1=false;
            if(last≥0) f1=true; //说明使用上一段
            last = nums[i] + max(last,0); //last负责新开的一段是否连带之前一段
            bool f2=false;
            if(maxv<last) f2=true; //说明加上当前段比较大 更改答案
            maxv = max(maxv,last); //maxv负责让当前一段与过去的一段做比较

            if(f1) {
                lastve.push_back(nums[i]);
            }else if(!f1){
                v.push_back(nums[i]);
                lastve=v;
            }
            if(f2){
                res=lastve;
            }
        }
        for(auto x:res) cout<<x<<" ";
        return maxv;
    }
};
```

```
//分治理 空间O(NlogN) 时间O(n) (注意线段树建树O(n),单次查询O(logN))
class Solution {
public:
    struct Node{
        int sum,s,ls,rs; //总和, 最大子数组和,最大前缀, 最大后缀
    };

    Node build(vector<int>&nums, int l, int r){ //表示对l,r范围内建树
        if(l == r) {
            return {nums[l], nums[l], nums[l], nums[l]};
        }
        int mid = l + r >> 1;
        auto L = build(nums,l,mid), R = build(nums,mid+1,r);
        //合并左右信息
        Node res;
        res.sum = L.sum + R.sum;
        res.s = max(max(L.s,R.s),L.rs+R.ls);
        res.ls = max(L.ls, L.sum+R.ls);
        res.rs = max(R.rs, R.sum+L.rs);
        return res;
    }

    int maxSubArray(vector<int>& nums) {
        auto t = build(nums,0,nums.size()-1);
        return t.s;
    }
};
```

## 56. 合并区间

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& a) {
        vector<vector<int>> res;
        if(a.empty()) return res;

        sort(a.begin(),a.end());

        int l=a[0][0],r=a[0][1];
        for(int i=1;i<a.size();i++){
            if(a[i][0] > r){
                res.push_back({l,r});
                l=a[i][0],r=a[i][1];
            }else{
                r=max(r,a[i][1]);
            }
        }
        res.push_back({l,r});
        return res;
    }
};
```

## 189. 轮转数组

```
//原地算法
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        if(nums.size()==1) return;
        k%=nums.size();
        reverse(nums.begin(),nums.end()); //先整体调整

        reverse(nums.begin(),nums.begin()+k); //前k个换一下啊

        reverse(nums.begin()+k,nums.end()); //后面的换一下
    }
};
```

## 238. 除自身以外数组的乘积

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n=nums.size();
        vector<int> l(n,1), r(n,1);

        for(int i=1;i<n;i++){ //前缀和
            l[i]=nums[i-1]*l[i-1];
        }

        for(int i=n-2;i>=0;i--){
            r[i]=nums[i+1]*r[i+1];
        }

        vector<int> res;
        for(int i=0;i<n;i++){
            if(i==0) res.push_back(r[i]);
            else if(i==n-1) res.push_back(l[i]);
            else res.push_back(l[i]*r[i]);
        }

        return res;
    }
};
```

C++

## 209. 长度最小的子数组

C++

```
class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        int v=1;
        int n=nums.size();

        for(auto &x:nums){
            if(x ≤ 0){ // 需要排除小于等于0的数的影响 因为需要修改出现的数为负数
                x=nums.size()+1;
            }
        }

        for(auto &x:nums){
            auto t=abs(x); // 刚需 因为可能后面的值已经是负数 过不了下面判断
            if(t ≤ nums.size() && t ≥ 1){
                nums[t-1]=-abs(nums[t-1]); // 扫一遍 如果这个数在1-n之间那么abs()之中就
                是负数的值
            }
        }

        for(int i=0;i<nums.size();i++){
            if(nums[i]>0){
                // 出现的第一个值仍然是正数的数 说明没有被修改成正数 是缺少的数。他的下标+1就
                是答案
                return i+1;
            }
        }

        return nums.size()+1;
    }
};
```

## 73. 矩阵置零

JAVA

```
class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
        int flag_col0 = false, flag_row0 = false; // 需要确认第一行第一列有0 不是被别人
        改的 而是自己本身就有 后续修改第一行列为0
        for (int i = 0; i < m; i++) {
            if (!matrix[i][0]) { // 第一行
                flag_col0 = true;
            }
        }
        for (int j = 0; j < n; j++) {
            if (!matrix[0][j]) { // 第一列
                flag_row0 = true;
            }
        }
        // 开始处理 i, j ≥ 1 的情况 为了复杂度先给第一行列弄成0先, 在对剩下的做修改
        for (int i = 1; i < m; i++) { // 如果(i, j) 等于0 那么对应的第一行第一列进行标记
            for (int j = 1; j < n; j++) {
                if (!matrix[i][j]) {
                    matrix[i][0] = matrix[0][j] = 0;
                }
            }
        }

        for (int i = 1; i < m; i++) { // 根据第一行列修改 对应(i, j)
            for (int j = 1; j < n; j++) {
                if (!matrix[i][0] || !matrix[0][j]) {
                    matrix[i][j] = 0;
                }
            }
        }
        // 根据第一行第一列修改 所有行列
        if (flag_col0) {
            for (int i = 0; i < m; i++) {
                matrix[i][0] = 0;
            }
        }
        if (flag_row0) {
            for (int j = 0; j < n; j++) {
                matrix[0][j] = 0;
            }
        }
    }
};
```

## 54. 螺旋矩阵

C++

```
//多个for循环方式
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        vector<int> res;
        if(matrix.empty()) return res;
        int n=matrix.size();
        int m=matrix[0].size();

        int top=0,bot=n-1,left=0,right=m-1;

        while(true){
            for(int i=left;i<=right;i++) res.push_back(matrix[top][i]);
            ++top;
            if(top>bot) break;
            for(int i=top;i<=bot;i++) res.push_back(matrix[i][right]);
            --right;
            if(right<left) break;
            for(int i=right;i>=left;i--) res.push_back(matrix[bot][i]);
            --bot;
            if(bot<top) break;
            for(int i=bot;i>=top;i--) res.push_back(matrix[i][left]);
            ++left;
            if(left>right) break;
        }
        return res;
    }
};
```

C++

```
//单个for方式 需要判重数组 方向数组
//要求 $i \leq n \times m$  当走到边界或者走过的时候 进行 $(d+1) \% 4$  重新设置方向
```

## 48. 旋转图像

C++

```
//找规律 发现先做沿着对角线翻转 然后再左右翻转
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n=matrix.size();
        for(int i=0;i<n;i++){
            for(int j=0;j<i;j++){
                swap(matrix[i][j],matrix[j][i]);
            }
        }

        for(int i=0;i<n;i++){
            for(int j=0,k=n-1;j<k;j++,k--){
                swap(matrix[i][j],matrix[i][k]);
            }
        }
    }
};
```

//找规律 每次向里面收缩，然后三个swap交换四个位置

```
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int up = 0, down = n-1;
        int left = 0, right = n-1;
        while(true){
            for(int i = 0; i < down-up; i++){ //两边相差的距离
                swap(matrix[up+i][left], matrix[down][left+i]);
                swap(matrix[down][left+i], matrix[down-i][right]);
                swap(matrix[down-i][right], matrix[up][right-i]);
            }
            //向里面收缩一波
            up++;
            down--;
            left++;
            right--;
            //到了边界
            if(up > down) break;
        }
    }
};

//i=1 要求顺时针旋转90 只要逆时针旋转三次
//matrix[up+i][left]  1 0 左中间
//matrix[down][left+i] 2 1 下变中间
//matrix[down-i][right] 1 2 右中间
//matrix[up][right-i]  0 1 上中间
//1 2 3 → 7 4 1
//4 5 6    8 5 2
//7 8 9    9 6 3
```

## 240. 搜索二维矩阵 II

C++

```
//从右上角开始找 大了往下 小了往左
class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int m = matrix.size(), n = matrix[0].size();
        int x = 0, y = n - 1; //x行 y是列
        while (x < m && y ≥ 0) {
            if (matrix[x][y] == target) {
                return true;
            }
            if (matrix[x][y] > target) { //说明太大 要往左走 左边是小的
                --y;
            }
            else { //说明太小 要变大 所以往下走
                ++x;
            }
        }
        //因为每次都是符合规定所以 能既找到大值又找到小值
        return false;
    }
};
```

### 链表题

#### 一些做题的基础，需要非常熟练：

- 1.进入函数就做判空
- 2.反转链表:非递归，需要pre指针和next指针，while(cur) 最终return pre. (递归方法容易忘，貌似也会慢一点)

```

//pre是null while条件是head while里面四句话 最后返回pre
ListNode* reverseList(ListNode* head) {
    if(!head) return head;
    ListNode* pre=nullptr;
    while(head){//如果传入一个左必右开的尾指针 这里就是head≠tail
        auto next=head→next; //先取出后面一个

        head→next=pre;
        pre=head;
        head=next;
    }
    return pre; //返回翻转后的最后一个点 也就是原节点
}

//
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if(!head||!head→next) return head; //记得!head→next前需要!head判空
        auto next=head→next;
        auto newhead=reverseList(next);
        next→next=head; //此时两个指针都指向next 下面一定head.next要设置为null
        head→next=nullptr;
        return newhead;
    }
};

```

### 3.快慢指针找中间节点

```

//偶数节点时返回中间节点左边 (如果需要返回右边节点走next就行)
ListNode* findMid(ListNode* head) {
    ListNode* fast = head;
    ListNode* slow = head;
    while (fast→next && fast→next→next ) { //注意是fast下一个 和 下下个.如果是
        返回偏右边节点这里就是fast&&fast→next (遇到危险直接返回)
        slow = slow→next;
        fast = fast→next→next;
    }
    return slow;
}

```

### 4.判断是否存在环

```

ListNode *detectCycle(ListNode *head) {
    if(!head||!head->next) return NULL;
    ListNode* fast=head;
    ListNode* slow=head;
    while(fast){
        fast=fast->next;slow=slow->next;
        if(!fast) return NULL;

        fast=fast->next;
        if(!fast) return NULL;

        if(slow==fast) {//有环 可以直接return
            fast=head; //下面找入环点 fast重回开头
            while(slow!=fast){
                slow=slow->next;
                fast=fast->next;
            }
            return fast;
        }
    }
    return NULL;
}

```

## 5.合并两条链表

//排序

```

ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    if(!list1) return list2;
    if(!list2) return list1;
    if(list1->val<=list2->val){
        list1->next=mergeTwoLists(list1->next,list2);
        return list1;
    }
    list2->next=mergeTwoLists(list2->next,list1);
    return list2;
}

```

//无排序

```

void merge(ListNode* l1,ListNode* l2){
    while(l1!=NULL && l2!=NULL){
        ListNode *n1=l1->next,*n2=l2->next; //记录要走的值
        l1->next=l2,l2->next=n1;
        l1=n1,l2=n2;
    }
}

```

6.凡是头结点可能被删除的,修改的,最好都建立一个虚拟头结点。

7.归并思想,先拆,再排序

## 160. 相交链表

//找链表相交的节点

//两个指针分别从两条链走到尽头后开始重新走  $a+b+c = c+a+b$  可以把相交的这一段b看成终点，问终点在哪里

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        if (headA == NULL || headB == NULL) return NULL;
        ListNode* pA = headA;
        ListNode* pB = headB;
        while(pA!=pB){
            pA=(pA==NULL)?headA:pA->next;
            pB=(pB==NULL)?headB:pB->next;
        }
        return pA;
        //走到尽头重新开始，若相交，c是共同点。链表A: a+c, 链表B : b+c.
        //a+c+b+c = b+c+a+c 。走了两遍c 则会在公共处c起点相遇。
        //若不相交, a +b = b+a 。因此相遇处是NULL
    }
};
```

## 234. 回文链表

// 快慢指针找到中间节点 并反转后半部分链表 然后同时一起走

C++

## 141. 环形链表

### 2. 两数相加

C++

```
//从左到右添加
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        auto dummy= new ListNode(),cur=dummy; //dummy虚拟头结点压根不会用，cur为当前处理到的新链表节点
        int t=0;
        while(l1||l2||t){ //关键
            if(l1) t+=l1->val,l1=l1->next;
            if(l2) t+=l2->val,l2=l2->next;
            cur->next=new ListNode(t%10);
            t/=10;
            cur=cur->next;
        }
        return dummy->next;
    }
};
```

## 19. 删除链表的倒数第 N 个结点

C++

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int& n) { //只有头结点 删除倒数第n个节点
        auto dummy=new ListNode(-1);
        dummy->next=head;

        int len=0;
        for(auto p=dummy;p;p=p->next) len++; //找长度

        auto p=dummy;
        for(int i=0;i<len-n-1;i++) p=p->next; //找倒数
        p->next=p->next->next;

        return dummy->next;
    }
};
```

## 24. 两两交换链表中的节点

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        if(head==nullptr||head->next==nullptr) return head;
        auto dummy=new ListNode(0);
        dummy->next=head;

        for(auto pre=dummy;pre->next&&pre->next->next;){ //确保交换的两个元素存在
            auto cur=pre->next,next=cur->next; //需要三个指针方便操作
            pre->next=next;
            cur->next=next->next;
            next->next=cur;

            pre=cur;
        }
        return dummy->next;
    }
};
```

## 25. K 个一组翻转链表

C++

```
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        if(head==nullptr||head->next==nullptr) return head;
        auto tail = head;
        for (int i = 0; i < k; i++) { //一共走了k步所以k是尽头
            //剩余数量小于k的话, 则不需要反转。
            if (tail == nullptr) {
                return head; //发现居然遍历到尾节点 说明不够 直接返回
            }
            tail = tail->next;
        }
        auto newhead= reverse(head,tail); //翻转这k个元素获得翻转后新的头节点 3, 4的话
        //这个值就是4
        head->next =reverseKGroup(tail,k); //递归处理之后的 原节点就是尾巴节点 和下一个
        //的连接 这个4

        return newhead;
    }

    ListNode* reverse(ListNode* head,ListNode*tail){
        ListNode* pre =NULL;
        while(head!=tail){ //左开右闭
            auto next = head->next;
            head->next=pre; //
            pre=head;
            head=next;
        }

        return pre;
    }
};
```

## 138. 随机链表的复制

```
class Solution {
public:
    Node* copyRandomList(Node* head) {
        if (head == nullptr) {
            return nullptr;
        }
        //插入新链表
        for (Node* node = head; node ; node = node->next->next) { //因为插入了 所以
原链表下一个节点需要next两次
            Node* nodeNew = new Node(node->val);
            nodeNew->next = node->next; //复制节点插到原节点后面
            node->next = nodeNew;
        }

        for (Node* node = head; node ; node = node->next->next) {
            Node* nodeNew = node->next;
            nodeNew->random = (node->random != nullptr) ? node->random->next :
nullptr; //必须给个next否则指向的是就节点
        } //被随机的链表的节点的下一个位置就是新的随机链表节点

        //拆分链表
        Node* headNew = head->next;
        for (Node* node = head; node ; node = node->next) {
            Node* nodeNew = node->next; //先保存
            node->next = node->next->next; //这一步一定要做 让原节点指针指回下个节点
            nodeNew->next = (nodeNew->next != nullptr) ? nodeNew->next->next :
nullptr; //新节点的下一个是下一个原节点 要改成新节点
        }
        return headNew;
    }
};
```

## 148. 排序链表

```
class Solution {
public:
    ListNode* sortList(ListNode* head) {
        if(!head) {
            return head;
        }
        auto newhead=merge(head);
        return newhead;
    }
    ListNode* find(ListNode* head) {
        if(!head||!head->next) return head;
        auto slow= head;
        auto fast= head;
        while(fast->next&&fast->next->next) {
            fast=fast->next;
            slow=slow->next;
            fast=fast->next;
        }

        return slow;
    }
    ListNode* mergelist(ListNode* l1,ListNode* l2) {
        if(!l1) return l2;
        if(!l2) return l1;
        if(l1->val<=l2->val) {
            l1->next=mergelist(l1->next,l2);
            return l1;
        } else {
            l2->next=mergelist(l1,l2->next);
            return l2;
        }
    }

    ListNode* merge(ListNode* head) {
        if(!head||!head->next) return head; //这条链只有一个 直接返回

        auto mid =find(head);
        auto nexthead = mid->next;
        mid->next=nullptr;

        head=merge(head);
        nexthead=merge(nexthead);

        return mergelist(head,nexthead);
    }
};
```

## 23. 合并 K 个升序链表

```
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        if(lists.size()==0) return nullptr;
        return merge(lists,0,list.size()-1);
    }
    ListNode* merge(vector<ListNode*>& lists,int l,int r){
        if(l==r) return lists[l];
        int mid = l+r>>1;
        ListNode* l1=merge(lists,l,mid);
        ListNode* l2=merge(lists,mid+1,r);
        return mergeLists(l1,l2);
    }
    ListNode* mergeLists(ListNode* l1,ListNode* l2){
        if(!l1) return l2;
        if(!l2) return l1;
        if(l1->val< l2->val){
            l1->next=mergeLists(l1->next,l2);
            return l1;
        }
        l2->next=mergeLists(l1,l2->next);
        return l2;
    }
};
```

## LRU缓存

C++

```
struct node{
    int key,value;
    node* prev;
    node* next;
    node():key(0),value(0),prev(nullptr),next(nullptr){}
    node(int key,int value):key(key),value(value),prev(nullptr),next(nullptr){}
};

class LRUCache {
public:
    map<int,node*>m;
    int size=0;
    int capacity=0;
    node* head=new node();//关键是头尾指针
    node* tail=new node();

    LRUCache(int _capacity) {
        capacity=_capacity;
        head->next=tail;
        tail->prev=head;
    }

    int get(int key) {
        if(m.count(key)){
            auto linknode= m[key];
            movetohead(linknode);
            return linknode->value;
        }else{
            return -1;
        }
    }

    void put(int key, int value) {
        if(m.count(key)){
            auto linknode= m[key];
            linknode->value=value;
            movetohead(linknode);
        }else{
            auto newnode= new node(key,value);
            addtohead(newnode);
            m[key]=newnode;
            ++size;
            if(size>capacity){
                auto p=tail->prev;
                deletenode(p);
                --size;
                m.erase(p->key); //map里面也要移除
                delete p; //这里因为是永久删除而不是移动所以需要 特殊去写
            }
        }
    }

    void deletenode(node* node){ //delete节点函数和
```

```

        node->next->prev=node->prev;
        node->prev->next=node->next;
    }
    void movetohead(node* node){
        deletenode(node);
        addtohead(node);
    }
    void addtohead(node* node){
        node->next=head->next;
        node->prev=head;
        head->next->prev=node;
        head->next=node;
    }

};

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache* obj = new LRUCache(capacity);
 * int param_1 = obj->get(key);
 * obj->put(key,value);
 */

```

## 树

### 基础

- 1.前中后序遍历。满足二叉搜索树=中序遍历结果由小到大
- 2.层序遍历，深序遍历
- 3.dfs的思想：

```

int dfs(TreeNode* root){
    if(!root) return 0;
    auto l=dfs(root->left),r=dfs(root->right);
    ans=max(ans,l+r);
    return max(l,r)+1;
}

```

MATLAB

- 4.根据数组建树

```

TreeNode* build(vector<int>&nums,int l, int r){
    if(l>r) return nullptr;
    int mid = l+r>>1;
    auto root = new TreeNode(nums[mid]);
    root->left=build(nums,l,mid-1);
    root->right=build(nums,mid+1,r);
    return root;
}

```

C++

## 226. 翻转二叉树

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:

    TreeNode* invertTree(TreeNode* root) {
        if(root==nullptr) {
            return nullptr;
        }
        //下面三句是将当前节点的左右子树交换 复杂度较高
        swap(root->left,root->right);
        //递归交换当前节点的 左子树
        invertTree(root->left);
        //递归交换当前节点的 右子树
        invertTree(root->right);

        return root;

    }

};
```

## 101. 对称二叉树

```
class Solution {
public:

    bool isSymmetric(TreeNode* root) {
        if(!root) return false;
        return dfs(root->left,root->right);
    }

    bool dfs(TreeNode* left , TreeNode* right){
        if(!left&&!right) return true;
        if(!left||!right||right->val!=left->val) return false;
        return dfs(left->left,right->right)&&dfs(left->right,right->left);
    }

};
```

## 114. 二叉树展开为链表

```
class Solution {
public:
    void flatten(TreeNode* root) {
        while(root){
            auto p=root->left;
            if(p){
                while(p->right) p=p->right; //如果root有左子树，就把右子树放到左子树的
                最右节点的右边
                p->right=root->right;
                root->right=root->left; //然后把左子树放到右子树
                root->left=nullptr;
            }
            root = root->right; //跳过去
        }
    }
};
```

## 105. 从前序与中序遍历序列构造二叉树

C++

```
class Solution {
public:
    TreeNode* myBuildTree(const vector<int>& preorder, const vector<int>&
inorder) {
        int n=preorder.size();

        int rootval=preorder[0];
        int i;
        for( i=0;i<n;i++){
            if(inorder[i]==rootval) break; //找到中间节点 如果不用i找 就需要通过中序计算
前序的长度
        }

        auto root=new TreeNode(rootval);
        if(n==1) return root;

        // 前序遍历确定根节点 中序遍历确定左右边界子树有多少个

        //把左右 前序中序数组找出来
        vector<int>leftpre(preorder.begin()+1,preorder.begin()+1+i);
        vector<int>leftin(inorder.begin(),inorder.begin()+i);

        vector<int>rightpre(preorder.begin()+i+1,preorder.end());
        vector<int>rightin(inorder.begin()+i+1,inorder.end());

        root->left=buildTree(leftpre,leftin);
        root->right=buildTree(rightpre,rightin);
        return root;
    }

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        int n = preorder.size();
        // 构造哈希映射，帮助我们快速定位根节点
        if(n==0)return nullptr;
        return myBuildTree(preorder, inorder);
    }
};
```

C++

```

class Solution {
public:
    unordered_map<int,int>pos;
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        for(int i=0;i<inorder.size();i++) pos[inorder[i]] = i;
        return build(preorder,inorder,0,preorder.size()-1,0,inorder.size()-1);
    }
    TreeNode* build(vector<int>&preorder,vector<int>&inorder,int pl,int pr,int
il,int ir){
        if(pl > pr) return nullptr;
        auto root = new TreeNode(preorder[pl]);
        int k= pos[root->val];
        root->left = build(preorder,inorder,pl+1,pl+1+(k-1-il),il,k-1);
        root->right = build(preorder,inorder,pl+1+(k-1-il)+1,pr,k+1,ir); //通过中
序数组计算前序数组的位置
        return root;
    }
};

```

## 437. 路径总和 III

C++

```

class Solution {
public:
    unordered_map<long long, int> prefix; //不从根节点开始, 但必须一条路径 说明使用前缀
和
    int res;
    void dfs(TreeNode *root, long long curr, int targetSum) {
        if (!root) return ;

        curr += root->val; //cur 根节点到这里的值

        //未知的x是题目要求的那一段满足要求的值的另一半。      x+ 满足目标值的一段 =根节点到
这个点的路径的值
        //求有多少个x就是求有多少个满足目标值的这一段
        res += prefix[curr - targetSum]; //表示遍历当前点的满足的数目

        prefix[curr]++;
        dfs(root->left, curr, targetSum);
        dfs(root->right, curr, targetSum);
        prefix[curr]--;
    }

    int pathSum(TreeNode* root, int targetSum) {
        prefix[0] = 1; //根节点开始遍历到的节点 = 所求的值也要算上 所以+1
        dfs(root, 0, targetSum);
        return res;
    }
};

```

## 236. 二叉树的最近公共祖先

```
class Solution {
public:
    int ans=0;
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        return dfs(root,p,q);
    }
    TreeNode* dfs(TreeNode* root,TreeNode* p,TreeNode* q){
        if(!root||root==p||root==q) return root;
        //如果找到了p或者q 那么就返回。找不到返回为空
        TreeNode* left=dfs(root->left,p,q);
        TreeNode* right=dfs(root->right,p,q);
        if(!left&&!right) return NULL;
        if(!left) return right;
        if(!right) return left;
        return root; //只有当left 或者 right的值都不为空的时候 也就是p和q的时候 这个值就是
        //对应的。否则说明不在这个子树里面
    }
};
```

## 124. 二叉树中的最大路径和

```
class Solution {
public:
    int maxx=INT_MIN;
    int maxPathSum(TreeNode* root) {
        if(!root) return 0;
        dfs(root);
        return maxx;
    }
    int dfs(TreeNode* root){ //从底向上
        if(!root) return 0;
        // cout<<root->val<<" ";
        int left=max(0,dfs(root->left));
        int right=max(0,dfs(root->right));
        maxx=max(maxx,root->val+left+right); //求答案的时候，表示以root为根节点的树的最大路径和
        return root->val+max(left,right); //dfs函数只返回本节点往下走一条最长的路
    }
};
```

C++

## 200. 岛屿数量

C++

```
//bfs版本 dfs更简单
class Solution {
public:
    int dx[4]={1,0,-1,0},dy[4]={0,1,0,-1};

    int res=0;
    int n=0,m=0;
    int numIslands(vector<vector<char>>& grid) {
        n=grid.size(),m=grid[0].size();

        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){

                if(grid[i][j]=='1')
                {
                    bfs(i,j,grid);
                    res++;
                    grid[i][j]='0';
                }
            }
        }
        return res;
    }
    void bfs(int x,int y,vector<vector<char>>& grid){
        queue<pair<int,int> >q;
        q.push({x,y});
        while(q.size()){
            auto t=q.front();
            q.pop();
            for(int i=0;i<4;i++){
                int a=t.first+dx[i],b=t.second+dy[i];

                if(a<0||a>=n||b<0||b>=m||grid[a][b]!='1') continue;

                grid[a][b]='0';
                q.push({a,b});
            }
        }
    }
};
```

## 994. 腐烂的橘子

C++

```
class Solution {
public:
    #define pii pair<int,int>
    int orangesRotting(vector<vector<int>>& g) {
        int n=g.size(),m=g[0].size();
        queue<pii>q;
        for(int i=0;i<n;i++){//多源bfs
            for(int j=0;j<m;j++){
                if(g[i][j]==2){
                    q.push({i,j});
                }
            }
        }

        int dx[]={1,0,-1,0},dy[]={0,1,0,-1};
        int res=0;
        if(q.size()) res--; //因为求的是层数-1

        while(q.size()){
            res++;
            auto sz=q.size();
            while(sz--){
                auto t=q.front();
                q.pop();
                for(int i=0;i<4;i++){
                    int x=t.first+dx[i],y=t.second+dy[i];
                    if(x<0 || x>=n || y<0 || y>=m || g[x][y] != 1) continue;
                    q.push({x,y});
                    g[x][y]=2;
                }
            }
        }
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                if(g[i][j]==1) return -1; //如果还有没有腐烂的
            }
        }
        return res;
    }
};
```

## 207. 课程表

C++

```
class Solution {
public:

    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<int>v(numCourses),d(numCourses);
        unordered_map<int,vector<int>>m;

        for(auto x:prerequisites){//拓扑排序 统计入度出度
            d[x[0]]++;
            m[x[1]].push_back(x[0]);
        }

        queue<int>q;
        for(int i=0;i<numCourses;i++){
            if(d[i]==0){//入度为0
                q.push(i);
            }
        }

        while(q.size()){
            auto t=q.front();
            q.pop();
            v[t]++;
            for(auto x:m[t]){//枚举后继节点
                d[x]--;
                if(d[x]==0)
                    q.push(x);
            }
        }

        for(int i=0;i<numCourses;i++){
            if(!v[i]){
                return false;
            }
        }
        return true;
    }
};
```

## 208. 实现 Trie (前缀树)

C++

```
class Trie {
public:
    struct Node{ //注意写节点就好
        Node* son[26]; //指针
        bool isend=false; //判断是否结束
        Node(){
            for(int i=0;i<26;i++){
                son[i]=nullptr; //必须赋初值
            }
        }
    };

    Node* root;

    Trie() {
        root = new Node();
    }

    void insert(string word) {
        auto p=root;
        for(auto c:word){
            int u=c-'a';
            if(!p->son[u]){
                p->son[u]=new Node();
                p=p->son[u];
            }else{
                p=p->son[u];
            }
        }
        p->isend=true;
    }

    bool search(string word) {
        auto p=root;
        for(auto c:word){
            if(p->son[c-'a']){
                p=p->son[c-'a'];
            }else{
                return false;
            }
        }
        if(p->isend) return true;
        return false;
    }

    bool startsWith(string prefix) {
        auto p=root;
        for(auto c:prefix){
            if(p->son[c-'a']){
                p=p->son[c-'a'];
            }
        }
    }
};
```

```

        }else{
            return false;
        }
    }

    return true;
}

};

/**
 * Your Trie object will be instantiated and called as such:
 * Trie* obj = new Trie();
 * obj->insert(word);
 * bool param_2 = obj->search(word);
 * bool param_3 = obj->startsWith(prefix);
 */

```

## 回溯

### 22. 括号生成

```

class Solution {
public:
    vector<string>res;
    vector<string> generateParenthesis(int n) {
        dfs(0,0,n,"");
        return res;
    }
    void dfs(int l,int r,int n,string s){
        if(l==n&& r==n)res.push_back(s);
        if(l<n) dfs(l+1,r,n,s+"(");
        if(r<n&&l>r) dfs(l,r+1,n,s+")");
    }
};

```

## 131. 分割回文串

C++

```
class Solution {
public:
    vector<vector<string>> res;
    vector<string> v;
    void dfs(string &s,int startval){//已经处理到startval
        if(startval ≥ s.size()){
            res.push_back(v);
            return ;
        }
        for(int i=startval;i<s.size();i++){//枚举start-i这段
            if(is(s,startval,i)){//如果这段是
                string sub=s.substr(startval,i-startval+1); //i-startval是长度
                v.push_back(sub); //取出这一段作为长度 继续处理后面的段
                dfs(s,i+1);
                v.pop_back();
            }
        }
    }
    bool is(string&s ,int start,int end){//start-end这范围是不是回文串 也可以dp进行预处理
        for(int i=start,j=end;i<j;i++,j--){
            if(s[i]≠s[j]) return false;
        }
        return true;
    }
    vector<vector<string>> partition(string s) {
        if(!s.size()) return res;
        dfs(s,0);
        return res;
    }
};
```

## 51.N皇后

C++

```
class Solution {
public:
    vector<vector<string>> res;
    vector<vector<string>> solveNQueens(int n) {
        vector<string> chess(n,string(n,'.')); // 设置一个备份
        dfs(n,0,chess);
        return res;
    }
    void dfs(int n,int row,vector<string>&chess){ // n总共多少行 row现在处理到第几行
        // ches整个棋盘
        if(row==n) {
            res.push_back(chess); // 满足了这么多行
            return ;
        }
        for(int i=0;i<n;i++){
            if(isvalue(row,i,chess)){ // chess棋盘的 第row行的第i个位置设置为q, 判断合法
                // 不
                chess[row][i]='Q';
                dfs(n,row+1,chess); // 下一行
                chess[row][i]='.';
            }
        }
    }
    bool isvalue (int nowrow,int nowcol,vector<string>& chess){ // 现在这个位置合不合法
        int n=chess.size();
        for(int i=nowrow-1;i>=0;i--){ // 看看
            if(chess[i][nowcol]=='Q') return false;
        }
        for(int i=nowrow-1,j=nowcol-1;i>=0&&j>=0;j--,i--){ // (i,j)左上角的这一列是不是已经设置过
            if(chess[i][j]=='Q') return false;
        }
        for(int i=nowrow-1,j=nowcol+1;i>=0&&j<n;i--,j++){ // (i,j)右上角的这一列是不是已经设置过
            if(chess[i][j]=='Q') return false;
        }
        return true;
    }
};

// 或者
class Solution {
public:
    int n;
    vector<bool> col,dg,udg;
    vector<string> path;
    vector<vector<string>> ans;
    vector<vector<string>> solveNQueens(int _n) {
        n=_n;
        col = vector<bool>(n);
        dg = udg = vector<bool>(n*2); // 用来表示对角线是否被占领
    }
};
```

```

        path = vector<string>(n,string(n,'.'));

        dfs(0); //表示第n行铺皇后
        return ans;
    }

    void dfs(int u){
        if(u==n){
            ans.push_back(path);
            return;
        }

        for(int i=0;i<n;i++){
            if(!col[i]&&!dg[u-i+n]&&!udg[u+i+n]){ //设置对角线为 u+i+n u-i+n
                col[i]=dg[u-i+n]=udg[u+i+n]=true;
                path[u][i]='Q';
                dfs(u+1);
                col[i]=dg[u-i+n]=udg[u+i+n]=false;
                path[u][i]='.';
            }
        }
    }
};

```

## 二分

### 注意

1.二分可以查单调的两段的分界点

## 74. 搜索二维矩阵

```

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        if(matrix.empty() || matrix[0].empty()) return 0;
        int n=matrix.size(),m=matrix[0].size();
        int l=0,r=n*m-1;
        while(l<r){
            int mid =l+r>>1;
            if(matrix[mid/m][mid%m] >= target) r=mid;
            else l=mid+1;
        }
        if(matrix[r/m][r%m]==target) return true;
        return false;
    }
};

```

## 4. 寻找两个正序数组的中位数

C++

```
//leetcode4: 寻找两个正序数组的中位数 (注释版)
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        int total = nums1.size() + nums2.size();
        if(total % 2 == 0){
            int left = findKthNumber(nums1, 0, nums2, 0, total/2);
            int right = findKthNumber(nums1, 0, nums2, 0, total/2 + 1);
            return (left + right)/2.0;
        }else{
            return findKthNumber(nums1, 0, nums2, 0, total/2 + 1);
        }
    }

    //递归函数: 找两正序数组中第K小的元素,i,j为起点下标 2个边界条件 注意第几个元素-1=下标
    int findKthNumber(vector<int> &nums1, int i, vector<int> &nums2, int j, int k){
        if(nums1.size() - i > nums2.size() - j) return findKthNumber(nums2, j,
nums1, i, k); //nums1.size() - i 才是剩余的长度
        if(nums1.size() == i) return nums2[j + k - 1]; //nums1已经被删成控
        if(k == 1) return min(nums1[i], nums2[j]); //求第1个元素, 则直接比较俩数组的起
点下标即可

        //idx表示这次找到的第k/2元素是数组的第几个元素 不是下标
        int idx1 = min(i + k/2, int(nums1.size())), idx2 = j + k/2; //数组1不一定
够k/2不要越界。数组2保持k/2没问题, 因为减少一点不影响
        if(nums1[idx1 - 1] < nums2[idx2 - 1]){
            //此时说明nums1的前半段没用, 直接后移k/2替换成新下标, k的长度整体减掉没用的那部
分长度
            return findKthNumber(nums1, idx1, nums2, j, k - (idx1 - i)); //不能换
成k/2 因为不一定数组一不一定够k/2
        }else{
            //此时说明nums2的前半段没用, 直接后移k/2替换成新下标, nums1不变, k的长度整体减
掉没用的那部分长度
            return findKthNumber(nums1, i, nums2, idx2, k - (idx2 - j)); //这里也
不能换成k/2
        }
    }
};
```

# 栈

## 20. 有效的括号

C++

```
class Solution {
public:
    bool isValid(string s) {
        stack<char>stk;
        for(auto &c:s){
            if(c=='(' || c=='[' || c=='{') stk.push(c);
            else{
                if(stk.size() && abs(stk.top()-c) <= 2) stk.pop();
                else return false;
            }
        }
        return stk.empty();
    }
};
```

## 394. 字符串解码

```
class Solution {
public:
    string decodeString(string s) {
        int u=0;
        return dfs(s,u);
    }
    string dfs(string&s,int&u){ //表示求从u开始的解码出的字符串
        string res; //答案字符串
        while(u<s.size() && s[u] != ']'){ //这里碰到有括号必须跳过 由外部函数处理, 因为进来的时候也是外部处理的
            if(s[u] >= 'a' && s[u] <= 'z' || s[u] >= 'A' && s[u] <= 'Z') res+=s[u++];
            else if (s[u] >= '0' && s[u] <= '9'){
                int k=u;
                while(s[k] >= '0' && s[k] <= '9') k++;
                int x=stoi(s.substr(u,k-u)); //字符串转成数字
                u=k+1; //跳过左括号
                string y=dfs(s,u);
                u++; //跳过右括号
                while(x-->0) res+=y; //增加x次字符串y
            }
        }
        return res;
    }
};
```

## 739. 每日温度

//单调栈

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n=temperatures.size();
        vector<int>ans(n);
        stack<int>s; // 栈里面存的是下标 对应的值是逐渐递低的
        // 因为出现高值（要求的值）的时候会消除前面所有的比他低值（栈里面）的答案 栈里只有越来越低值才能存进去
        for(int i=0;i<n;i++){
            while(!s.empty()&&temperatures[i]>temperatures[s.top()]){
                // 一旦发现 比栈顶元素大的值 说明出现了天气高的日子 通过下标修改对应的答案
                int pre=s.top();
                ans[pre] =i-pre;
                s.pop();
            }
            s.push(i);
        }
        return ans;
    }
};
```

## 84. 柱状图中最大的矩形

C++

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int n=heights.size();
        vector<int> left(n,-1),right(n,n); //次数的初始化为-1很重要。考虑计算一根柱子的
        面积
        stack<int> sta;

        //左边第一个比它矮的元素
        for(int i=0;i<n;i++){
            while(!sta.empty()&&heights[sta.top()]>=heights[i]) sta.pop();
            if(!sta.empty()) left[i]=sta.top(); //如果sta不为空，则此时的栈顶为左边第一
            个比它小的元素的下标
            sta.push(i);
        }

        //清空栈
        while(!sta.empty()) sta.pop();

        //右边第一个比它矮的元素
        for(int i=n-1;i>=0;i--){
            while(!sta.empty()&&heights[sta.top()]>=heights[i]) sta.pop();
            if(!sta.empty()) right[i]=sta.top(); //右边第一个比它小的元素的下标，如果没有
            有在找到默认为n
            sta.push(i);
        }

        //计算结果 结果为每根住的高度*左右两边比他矮的元素的下标之差
        int res=0;
        for(int i=0;i<n;i++){
            res=max(res,heights[i]*(right[i]-left[i]-1)); //左右比他小的距离 如图示是
            下标为2的柱子的高度*（下标4-下标1+1）
        }
        return res;
    }
};
```

## 215. 数组中的第K个最大元素

//快速算法 先排序后递归 Onlogn 快选On

```

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        return quick(nums, nums.size()-k, 0, nums.size()-1);
    }
    int quick(vector<int>& nums, int k, int l, int r){
        if(l==r) return nums[l];
        int i=l-1, j=r+1, x=nums[l];
        while(i<j){
            while(nums[++i]<x) ;
            while(nums[--j]>x) ;
            if(i<j){
                swap(nums[i], nums[j]);
            }
        }
        if(k<=j){
            return quick(nums, k, l, j);
        }else{
            return quick(nums, k, j+1, r);
        }

        // return
    }
};

```

## 215. 数组中的第K个最大元素

```

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        return quick(nums, nums.size()-k, 0, nums.size()-1);
    }
    int quick(vector<int>& nums, int k, int l, int r){ //在l-r区间内找到第k个元素
        if(l==r) return nums[l];
        int i=l-1, j=r+1, x=nums[l]; //边界问题
        while(i<j){
            while(nums[++i]<x) ;
            while(nums[--j]>x) ;
            if(i<j){
                swap(nums[i], nums[j]);
            }
        }
        if(k<=j){
            return quick(nums, k, l, j);
        }else{
            return quick(nums, k, j+1, r);
        }

        // return
    }
};

```

## 347. 前 K 个高频元素

C++

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        // 计数排序
        unordered_map<int,int>m;
        for(auto x:nums) m[x]++;
        int n=nums.size();
        vector<int>s(n+1);
        for(auto [x,c]:m) s[c]++;
        int i=n,t=0; // i表示出现次数 k是求第几个
        while(t<k) t+=s[i--]; // i停止的位置就是前k个最多到达的最多出现的次数
        vector<int>res;
        for(auto [x,c]:m)
            if(c>i) // 回去找到这些数字
                res.push_back(x);
        return res;
    }
};
```

## 贪心+dp

### 55. 跳跃游戏

C++

```
class Solution {
public:
    bool canJump(vector<int>& nums) {
        for(int i=0,j=0;i<nums.size();i++){
            if(j<i) return false;
            j=max(j,i+nums[i]);
        }
        return true;
    }
};
```

### 45. 跳跃游戏 II

```
class Solution {
public:

    int jump(vector & nums) {
        int n=nums.size();
        vector f(n); // f[i]是到达i需要的步数
        for(int i=1,j=0;i<n;i++){
            while(j+nums[j]<i) j++; // 题目保证一定可以到终点 当发现j跳不动i就让j++
            f[i]=f[j]+1; // 只在j基础上走一步
        }
        return f[n-1];
    }
};
```

## 763. 划分字母区间

C++

```
class Solution {
public:
    //需要划分出的片段每个字母只在其中一个片段中, 而且多
    vector<int> partitionLabels(string s) {
        vector<int> last(26), res;
        for(int i=0; i<s.size(); i++) last[s[i]-'a']=i; // 标记每个字母最后出现的位置
        int start=0, end=0; // 需要标记这段的起点终点
        for(int i=0; i<s.size(); i++){
            end=max(end, last[s[i]-'a']); // 中途便利到的字母 会影响这段的终点
            if(i==end){ // 已经顺利遍历到字母最后的终点 为了段数更多, 就尽快放入
                res.push_back(end-start+1);
                end=start=i+1;
            }
        }
        return res;
    }
};
```

## 118. 杨辉三角

C++

```
class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> res;
        for(int i=0; i<numRows; i++){
            vector<int> v;
            for(int j=0; j<=i; j++){
                if(j==0 || j==i) v.push_back(1);
                else v.push_back(res[i-1][j]+res[i-1][j-1]);
                if(j==i){
                    res.push_back(v);
                }
            }
        }
        return res;
    }
};
```

## 279. 完全平方数

C++

```
//不用数学定理
class Solution {
public:
    vector<int>f;
    int numSquares(int n) {
        vector<int>f(n+1);
        for(int i=1;i<=n;i++){
            int minv=INT_MAX;
            for(int j=1;j*j<=i;j++){
                minv = min(minv,f[i-j*j]); // x + 平方数 = 要求的数
            }
            //平方数是1
            f[i]=minv+1;
        }
        return f[n];
    }
};
```

## 零钱替换

C++

```
class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int>f(amount+1,1e8);
        f[0]=0;
        for(auto v:coins){
            for(int j=v;j<=amount;j++){//多重背包
                f[j]=min(f[j],f[j-v]+1);
            }
        }
        if(f[amount]==1e8) return -1;
        return f[amount];
    }
};
```

## 139. 单词拆分

C++

```
//用的dp复杂度较高 $O(N^3)$  也可以用字符串哈希把substr优化为 $O(1)$ 最终为 $O(N^2)$ 
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {

        auto wordDictSet = unordered_set<string> ();
        for (auto word: wordDict) {
            wordDictSet.insert(word);
        }
        auto dp = vector<bool> (s.size() + 1);
        for (int i = 0; i < s.size(); ++i) { //暴力 $O(N^2)$  dp[i]表示字符串到i能否被字典完
全表示
            if(wordDictSet.find(s.substr(0,i+1)) != wordDictSet.end()){
                dp[i]=true;continue;
            }
            for (int j = 0; j <= i; ++j) { //遍历前面的字符
                if(wordDictSet.find(s.substr(j+1, i - j)) != wordDictSet.end())
                    if (dp[j] && wordDictSet.find(s.substr(j+1, i - j)) !=
wordDictSet.end()){
                        //0-j这一段可以被表示 && 从j+1开始的子字符串是被表示 如果0-i i-j都
是true 那么i-j也能用来什么表示
                        dp[i] = true; //0-i
                        break; //可以不用继续看后面的
                    }
            }
        }

        return dp[s.size()-1];
    }
};
```

## 300. 最长递增子序列

C++

```
class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n=nums.size();
        vector<int>f(n+1); //维护一个递增的数组
        int len=0;
        for(int i=0;i<n;i++){
            int l=0,r=len;
            while(l<r){
                int mid =(l+r+1)>>1;
                if(f[mid]<nums[i]){
                    l=mid; //r找到元素小于mid的数的最右边 其右边的第一个数就是该放入的位置
                }else{
                    r=mid-1;
                }
            }
            f[r+1]=nums[i]; //
            len=max(len,r+1);
        }
        return len;
    }
};
```

## 152. 乘积最大子数组

C++

```
class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int n=nums.size();
        double res=INT_MIN;
        double imax=1; //用double不然longlong不够用
        double imin=1;
        for(int i=0;i<n;i++){
            if(nums[i]<0){
                swap(imax,imin);
            }
            imax=max(imax*nums[i],(double)nums[i]);
            imin=min(imin*nums[i],(double)nums[i]);
            res=max(res,imax);
        }
        return res;
    }
};
```

## 416. 分割等和子集

C++

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int n=nums.size(),m=0;
        for(auto x:nums) m+=x;
        if(m%2) return false;
        m/=2;
        vector<int> f(m+1);
        f[0]=1;
        for(auto x:nums)
            for(int j=m;j>=x;j--){//01背包
                f[j]=f[j-x];
            }
        return f[m];
    }
};
```

## 32. 最长有效括号

C++

```
class Solution {
public:
    int longestValidParentheses(string s) {
        stack<int>stk;
        //分段找最大合法括号序列 ()((())就算() 注意这种情况的答案是2
        //如果靠栈判断 你不知道你将来的( 有没有右括号 是不是t+2
        int res=0,t=0;
        for(int i=0,start=-1;i<s.size();i++){
            if(s[i]=='(') stk.push(i);
            else{
                if(stk.size()){
                    stk.pop();
                    if(stk.size()){
                        res=max(res,i-stk.top()); //针对()() 因为是pop之后再加所以直接减去
                    }
                }
                else{
                    res=max(res,i-start); //这里是解决((()) 每多一个右括号而且有左括号匹配就max多一个值。而不是靠+的方式
                }
            }
            else{
                start=i;
            }
        }
        return res;
    }
};
```

设计字符串的题目 最好让他从1开始 不用考虑太多边界问题

## 5. 最长回文子串

```
// 区间dp
class Solution {
public:
    string longestPalindrome(string s) {
        int n=s.size();
        vector<vector<int>>>f(n+1,vector<int>(n+1,0));
        s=" "+s;
        for(int i=1;i<=n;i++){
            f[i][i]=1;
        }
        int l=1,r=1;
        for(int len=2;len<=n;len++){//必须先枚举长度
            for(int i=1;i<=n;i++){
                int j=i+len-1;
                if(j<=n){
                    if(s[i]==s[j]){
                        if(len==2)
                            f[i][j]=1;
                        else {
                            f[i][j]=max(f[i][j],f[i+1][j-1]);
                        }
                        if(f[i][j]&& r-l+1<len){
                            l=i,r=j;
                        }
                    }
                }
            }
        }
        // cout<<l<<" "<<r<<endl;
        return s.substr(l,r-l+1);
    }
};
```

## 1143. 最长公共子序列

C++

```
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int n=text1.size(),m=text2.size();
        text1 = " "+text1;
        text2 = " "+text2;
        vector< vector<int> >f(n+1,vector<int>(m+1,0));
        for(int i=1;i≤n;i++){
            for(int j=1;j≤m;j++){
                //f[i][j]表示串1到i, 串2到j的时候最长公共子序列的长度
                f[i][j]=max(f[i-1][j],f[i][j-1]);
                f[i][j]=max(f[i][j],f[i-1][j-1]+(text1[i]==text2[j])); //求子序
                列!!! 不是编辑距离
            }
        }

        return f[n][m];
    }
};
```

## 72. 编辑距离

C++

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int n=word1.size(),m=word2.size();
        vector<vector<int>>f(n+1,vector<int>(m+1,0));
        word1=" "+word1;
        word2=" "+word2;
        for(int i=1;i≤n;i++) f[i][0]=i; //想前i个字符 和 前j个相同的时候最小的开发次数
        for(int j=1;j≤m;j++) f[0][j]=j;

        for(int i=1;i≤n;i++){
            for(int j=1;j≤m;j++){
                f[i][j]=min(f[i-1][j],f[i][j-1])+1;
                f[i][j]=min(f[i][j],f[i-1][j-1]+!(word1[i]==word2[j]));
            }
        }

        return f[n][m];
    }
};
```

## 75. 颜色分类

C++

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int n=nums.size();
        int p0=0,p2=n-1;
        for(int i=0;i<=p2;i++){//只for指针i, p0p1只有在换了内容的时候才会发生改变
            while(i<=p2 && nums[i]==2){//发现i的内容应该放到2里面 就交换 然后p往前走 不管p的内容是不是
                swap(nums[i],nums[p2]);
                --p2; //因为这里交换而i又没有动 所以不会出现死循环的情况
            }
            if(nums[i]==0){//只能交换一次
                swap(nums[i],nums[p0]);
                ++p0;
            }
        }
    }
};
```

## 31. 下一个排列

C++

```
//      ^           \           \
//     / \  ⇒      \ /  ⇒      \
//      \           V           \
//     2 3 1       3 1 2       3 2 1
```

// 1. 想要下一个降序 肯定要求变化最小 那么改变后面就是最后结果

// 2. 如果后面已经没办法的 那就找到第一个非降序的数 他的前一个就是要操作的数，操作这个数让数字变得更大一些

// 3. 跟他交换的数 要找一个 比这个数稍微大一点的数。让他们交换后，同时让降序变成升序，保持降序的空间 那么就肯定比之前的数要小 2

```
class Solution {
public:
    void nextPermutation(vector<int>& a) {
        int n=a.size();
        if(n<=1) return;
        int i;
        for(i=n-2;i>=0;i--){//123456 →123465→123546
            if(a[i]<a[i+1]){//从右到左 找到第一个比前面大的数
                //这个数的前面一个数就是对应需要调换的那个数i
                break;
            }
        }
        if(i>=0){
            int j;
            for(j=n-1;j>=i+1;j--){//在找到这个数的后面的
                //找到第一个比数字i大的最小数(因为这个数i的右边一定是从大到小的)
                if(a[j]>a[i]) break;
            }

            swap(a[i],a[j]);//希望影响最小所以改变后面的最大值和他做改变

            reverse(a.begin()+i+1,a.end());//此时剩下后面的为降序 重置为升序就好
        }
        //找不到说明已经是从小到大排序的了 下一个就是从小到大
        else{
            reverse(a.begin(),a.end());
        }
    }
};
```